UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Software Architectures

Practice 3

Zoltán Richárd JÁNKI, PhD

Example project: EventHub

Overview

- Quality Attribute Scenarios
  - Prevent double-booking (Performance/Security)
  - Payment gateway fails, hold/resume (Availability/Performance)
  - Block unauthorized admin access (Security/Availability)
  - Real-time sales data (Performance/Availability/Testability)
  - Refund reliability (Availability/Performance)
  - Scalability under peak load (Performance/Scalability)

Example project: EventHub

Overview

- Decision Matrix Criteria & Weights

Criterion	Weight
Concurrency & booking integrity	30
Payment failure resilience	20
Security & access control	20
Real-time data performance	10
Refund reliability	10
Scalability under peak load	10

Example project: EventHub

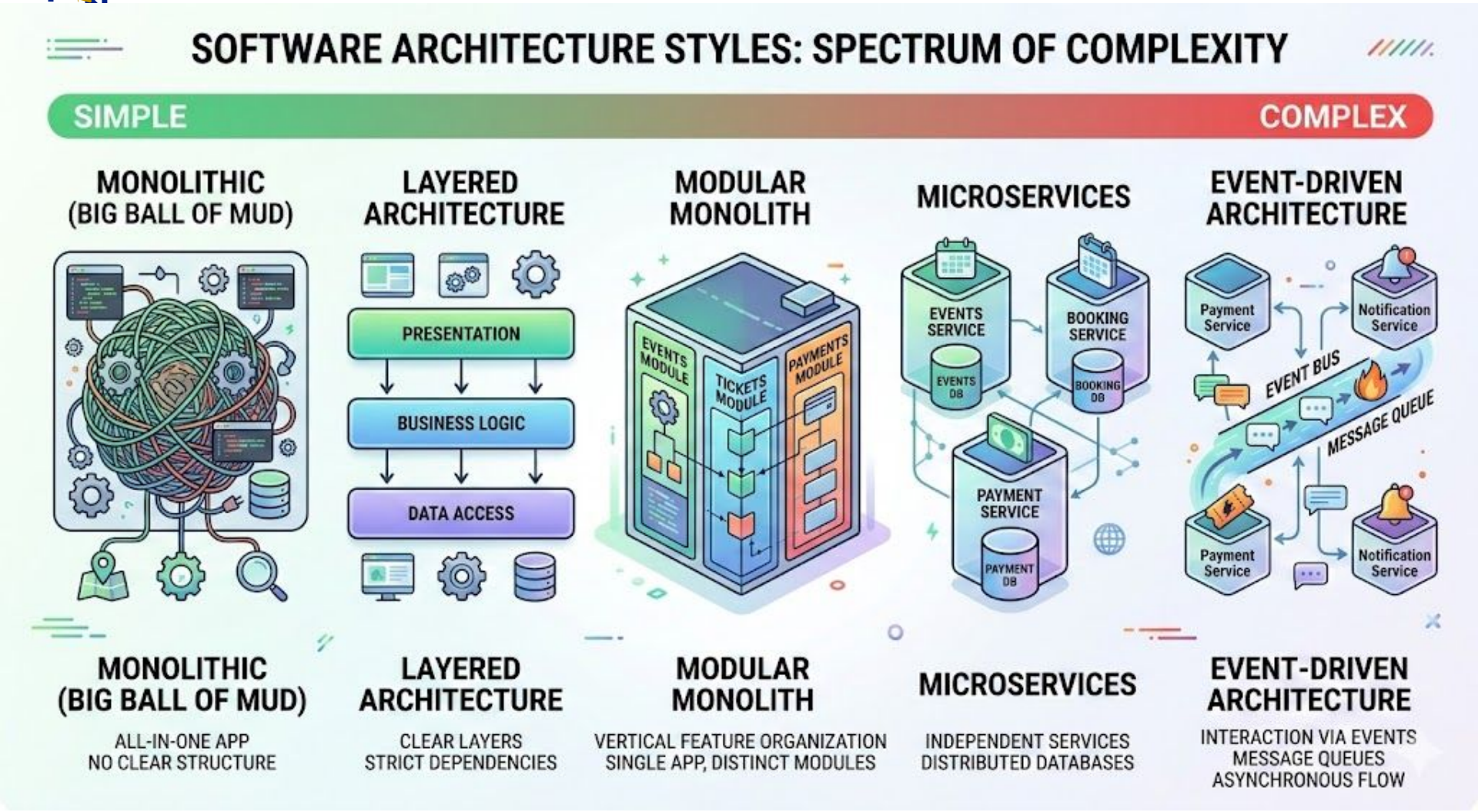
Overview

- Selected frameworks

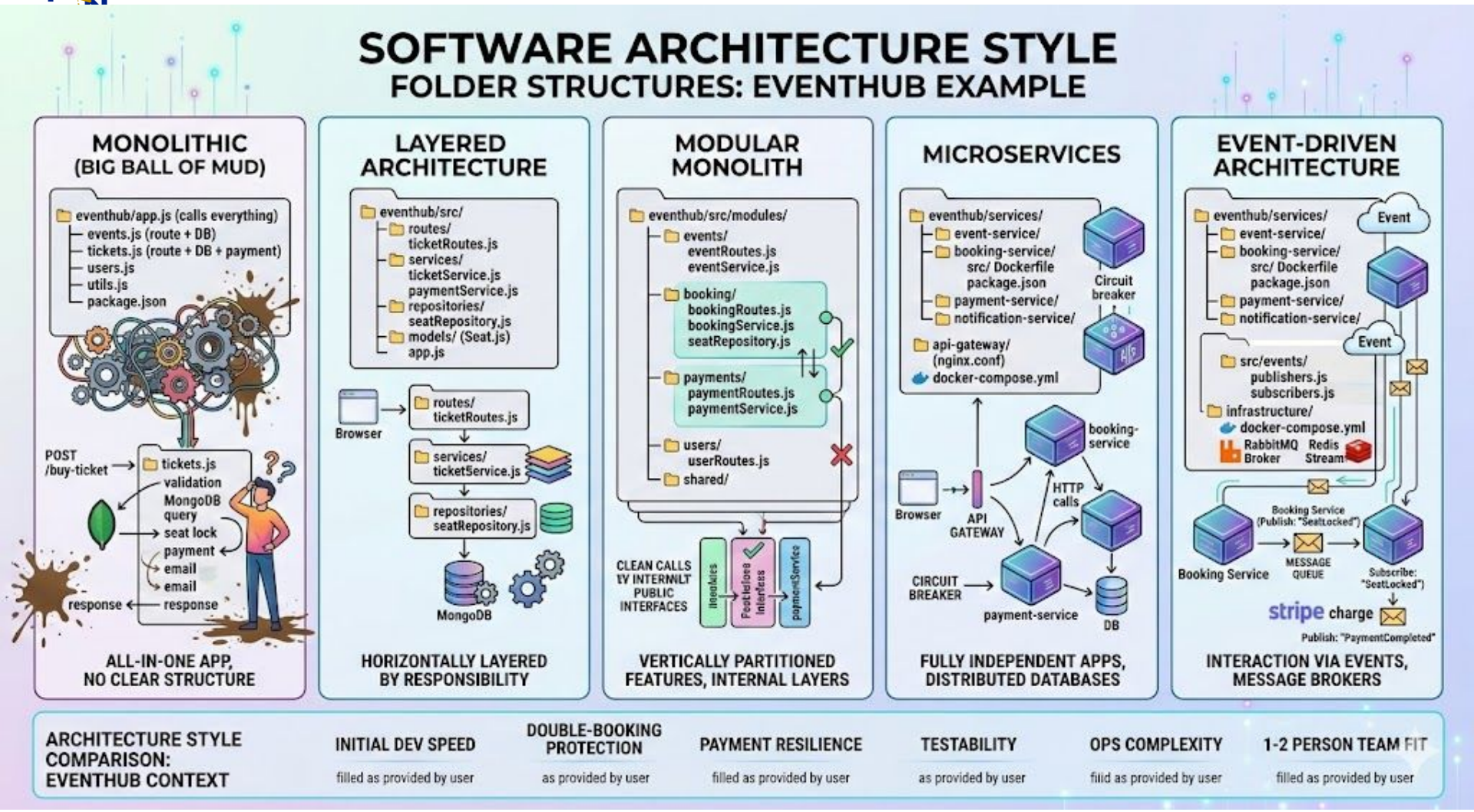
Technological solution	Weighted Total
Frontend	
Angular	**470**
React	450
Vue.js	310
Svelte	290
Backend	
Spring Boot	480
Node.js	**420**
Go	400
Django	380

Database	
Aurora	500
PostgreSQL	**480**
MySQL	380
MongoDB	360

Architecture styles



Architecture styles



Decision aspects

Which architecture style should be used for EventHub?

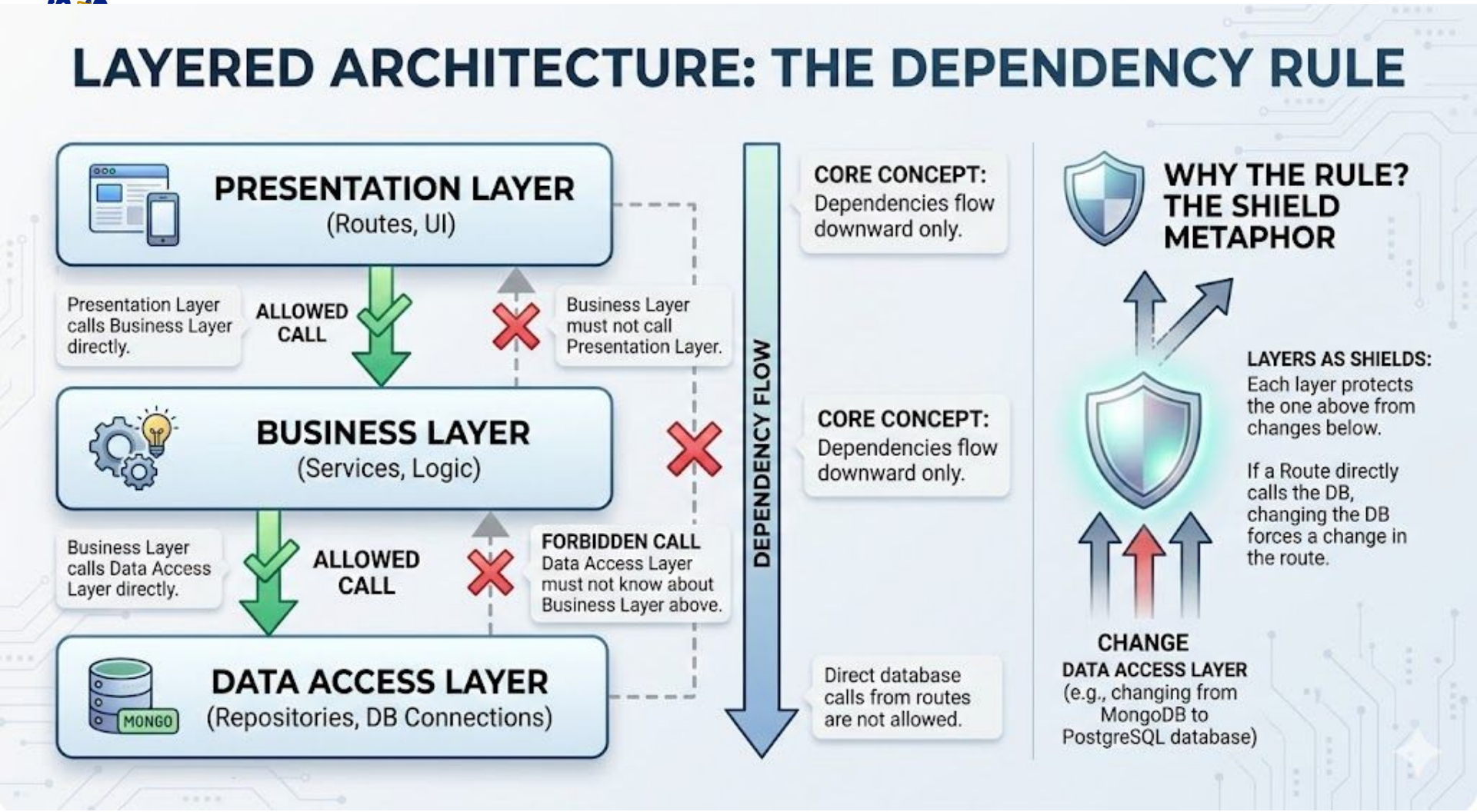
- Team size: 1-2 people
- Project phase: MVP
- Operational complexity: simple
- Key QAW drivers: booking integrity, payment resilience, security

Style	Team size	Project phase	Operational complexity	Key QAW drivers
Big Ball of Mud	✅ (fast)	✅ (fast)	✅ (simple)	❌ (unsustainable)
Layered	✅ (easy to learn)	✅ (enough for MVP)	✅ (simple)	✅ (separate responsibilities)
Modular Monolith	⚠️ (overengineered)	⚠️ (too much boilerplate)	✅ (simple)	✅ (good)
Microservices	❌ (overhead)	❌ (slow start)	❌ (complex)	✅ (best isolation)
Event-Driven	❌ (overhead)	❌ (overkill)	❌ (more complex)	✅ (best resilience)

Layered architecture

- Presentation Layer
  - Routes, controllers, request/response handling
- Business Logic Layer
  - Domain rules, validation
- Data Access Layer
  - Database queries

The dependency rule



Layer 1 - Presentation

- Responsibilities
  - Accepts HTTP requests
  - Extract input (req.body, req.params, req.query)
  - Return HTTP responses (mapping the service result)
- Not responsible for
  - database queries
  - business decisions
  - calling external APIs

Layer 2 - Business Logic (Services)

- Responsibility
  - Business rules and domain logic
  - Orchestration of multiple repository calls into a single operation
  - Calling other services
  - Domain-specific errors
- Not responsible for:
  - HTTP concepts (status codes, request objects, headers)
  - Direct database queries
  - Formatting responses

Layer 3 - Data Access (Repositories)

- Responsibility
  - CRUD operations
  - Query construction and optimization
- Not responsible for:
  - Business decisions
  - HTTP concepts
  - Calls to external APIs